COMS 6998: High Performance Machine Learning

Homework Assignment 3 — CUDA Matrix Operations, Unified Memory, and Convolution

Rajvardhan Patil

Spring 2026

Contents

1 Setup **2**

2 Part-A: CUDA Matrix Operations (40 points) **2**
- 2.1 Problem 1 — Vector Addition and Coalescing Memory Access . . . 2
  - 2.1.1 Baseline: `vecadd00` (`vecaddKernel00.cu`) . . . 2
  - 2.1.2 Optimised: `vecadd01` (`vecaddKernel01.cu`) . . . 2
  - 2.1.3 Q1 & Q2 Results . . . 3
- 2.2 Problem 2 — Shared CUDA Matrix Multiplication . . . 3
  - 2.2.1 Baseline: `matmult00` (`matmultKernel00.cu`) . . . 3
  - 2.2.2 Q4 Optimisation: `matmult01` (`matmultKernel01.cu`) . . . 3
  - 2.2.3 Q3 & Q4 Results . . . 3
  - 2.2.4 Q5 — Rules of Thumb for CUDA Performance . . . 4

3 Part-B: CUDA Unified Memory (20 points) **4**
- 3.1 Overview . . . 4
- 3.2 Q1 Results — CPU . . . 4
- 3.3 Q2 Results — GPU Without Unified Memory . . . 5
- 3.4 Q3 Results — GPU With Unified Memory . . . 5
- 3.5 Q4 — Performance Charts . . . 5
- 3.6 Analysis . . . 6

4 Part-C: Convolution in CUDA (50 points) **6**
- 4.1 Problem Setup . . . 6
- 4.2 C1 — Simple Convolution (no shared memory) . . . 6
- 4.3 C2 — Tiled Convolution (shared memory) . . . 7
- 4.4 C3 — cuDNN Convolution . . . 7
- 4.5 C4 — OpenAI Triton (Bonus) . . . 7
- 4.6 Program Output (`program_output.csv`) . . . 7
- 4.7 Performance Comparison . . . 8
- 4.8 Analysis . . . 8

5 Conclusion **8**

1 Setup

GCP VM, V100 GPU. The instance had CUDA 12.8 already installed but `nvcc` wasn't in PATH by default so I had to add `/usr/local/cuda/bin` manually before anything would compile. Full config:

Parameter	Value
Instance name	project1-v100
Machine type	n1-standard-8 (8 vCPUs, 30 GB RAM)
GPU	1 × NVIDIA Tesla V100-SXM2 (16 GB HBM2)
Driver version	570.211.01
CUDA version	12.8
cuDNN version	9.x
OS	Ubuntu 24.04 (Deep Learning VM image)
Compiler	nvcc 12.8, g++ 13.x
Zone	us-central1-c

For timing I used the `timer.cu` wrapper (`gettimeofday`-based) with one warm-up run before the timed run, and `cudaDeviceSynchronize()` before stopping. Part-B uses CUDA events instead — the timer.cu precision wasn't enough for the sub-millisecond cases.

2 Part-A: CUDA Matrix Operations (40 points)

2.1 Problem 1 — Vector Addition and Coalescing Memory Access

2.1.1 Baseline: `vecadd00` (`vecaddKernel00.cu`)

The baseline assigns each thread N *consecutive* array elements:

```
int blockStartIndex  = blockIdx.x * blockDim.x * N;
int threadStartIndex = blockStartIndex + threadIdx.x * N;
for (int i = threadStartIndex; i < threadStartIndex + N; i++)
    C[i] = A[i] + B[i];
```

Listing 1: Non-coalescing kernel (vecaddKernel00.cu)

Thread 0 works on elements 0 through $N-1$, thread 1 works on N through $2N-1$, and so on. Adjacent threads in a warp are N floats apart in memory, so each warp iteration generates N separate 4-byte transactions instead of one 128-byte coalesced one.

2.1.2 Optimised: `vecadd01` (`vecaddKernel01.cu`)

The fix is to stride by the total number of launched threads instead. Thread i processes indices i, $i + \text{totalThreads}$, $i + 2 \cdot \text{totalThreads}$, ..., so adjacent threads always hit adjacent addresses:

```
int totalThreads = gridDim.x * blockDim.x;
int globalIdx    = blockIdx.x * blockDim.x + threadIdx.x;
for (int i = 0; i < N; i++) {
    int idx = globalIdx + i * totalThreads;
    C[idx] = A[idx] + B[idx];
}
```

Listing 2: Coalescing kernel (vecaddKernel01.cu)

2.1.3 Q1 & Q2 Results

Table 1: Vector addition performance — GFLOPs/s (V100, GridWidth=60, BlockWidth=128)

ValuesPerThread	Vector size	vecadd00 (GFLOPs/s)	vecadd01 (GFLOPs/s)	Speedup
500	3,840,000	12.84	16.20	1.26×
1000	7,680,000	14.46	16.41	1.13×
2000	15,360,000	14.56	16.50	1.13×

The coalesced kernel is 13–26% faster. The V100's L2 cache is large enough to absorb a lot of the non-coalesced traffic at these sizes, which keeps the gap smaller than the theory alone would suggest.

2.2 Problem 2 — Shared CUDA Matrix Multiplication

2.2.1 Baseline: `matmult00` (`matmultKernel00.cu`)

Each thread computes one element of C. A 16×16 block covers a 16×16 output tile. Sub-matrices of A and B are cooperatively loaded into shared memory, synced, then used to accumulate:

```
__shared__ float shared_A[BLOCK_SIZE][BLOCK_SIZE];
__shared__ float shared_B[BLOCK_SIZE][BLOCK_SIZE];
// each thread loads one element of each shared sub-matrix
shared_A[thread_row][thread_col] = Asub[thread_row * A.stride + thread_col];
shared_B[thread_row][thread_col] = Bsub[thread_row * B.stride + thread_col];
__syncthreads();
for (int e = 0; e < BLOCK_SIZE; e++)
    Cvalue += shared_A[thread_row][e] * shared_B[e][thread_col];
```

Listing 3: Baseline matmul — one C value per thread

2.2.2 Q4 Optimisation: `matmult01` (`matmultKernel01.cu`)

Each thread now computes four output values — a 2×2 sub-tile of C. With `FOOTPRINT_SIZE=32` (via `-DFOOTPRINT_SIZE=32` in the Makefile), each 16×16 thread block now covers a 32×32 output region.

- Shared memory: `shared_A[32][16]`, `shared_B[16][32]`
- Each thread loads 2 rows from A_{sub} and 2 columns from B_{sub}
- Four accumulators `Cvalue[2][2]`, four stores at the end

2.2.3 Q3 & Q4 Results

Table 2: Matrix multiplication performance — GFLOPs/s on V100

Matrix size	matmult00	matmult01	Speedup
256×256	1234.5	1466.0	1.19×
512×512	2507.6	4124.2	1.64×
1024×1024	3020.5	5532.7	**1.83**×

At 256^2 both kernels are underutilizing the V100's 80 SMs so the gap is small. The benefit shows up more at 1024^2 where there's enough work to keep the hardware busy and the 32×32 footprint's extra shared-memory reuse actually matters. `#pragma unroll` on the inner accumulation loop also matters, though I'm less certain how much of the $1.83\times$ comes from that versus just the larger tile getting more reuse out of what's already in shared memory.

2.2.4 Q5 — Rules of Thumb for CUDA Performance

Coalesce global memory accesses — thread i should access index i, not $i \cdot N$. Use shared memory to avoid loading the same global data more than once per block. Warm up before timing; first launches are always slow. Unroll inner loops when the count is known at compile time. More work per thread reduces scheduling overhead, which is what the 4-value matmul shows.

Occupancy matters too, though it's harder to reason about without profiling — register usage and shared memory both feed into it and the interaction between them isn't something you can easily read off the code. When I've needed to care about it I've just tried a few block sizes and looked at what `ncu` reports. Warp divergence is worth knowing about; it didn't actually come up here.

3 Part-B: CUDA Unified Memory (20 points)

3.1 Overview

Three programs measure the time to compute $C = A+B$ for float vectors of size $K \in \{1, 5, 10, 50, 100\}$ million:

`q1` is CPU-only — plain `malloc`, single loop, timed with `clock_gettime(CLOCK_MONOTONIC)`.

`q2` uses `cudaMalloc` with explicit `cudaMemcpy` and tests three GPU configurations. Timing is kernel-only via CUDA events, not including the transfers.

`q3` uses `cudaMallocManaged` with the same three configurations. No `cudaMemcpy` — the driver migrates pages as needed.

The three GPU configurations:

1. 1 block, 1 thread
2. 1 block, 256 threads
3. $\lceil N/256 \rceil$ blocks, 256 threads per block

3.2 Q1 Results — CPU

Table 3: CPU-only vector addition time

K (millions)	Time (ms)
1	0.714
5	4.374
10	9.636
50	49.619
100	99.841

Linear scaling with K, as expected for a memory-bound loop.

3.3 Q2 Results — GPU Without Unified Memory

Table 4: GPU kernel time (ms) — explicit `cudaMalloc`/`cudaMemcpy`

K (M)	CPU (ms)	1blk/1thr (ms)	1blk/256thr (ms)	N-blks/256thr (ms)
1	0.714	65.4	1.443	0.020
5	4.374	301.0	6.650	0.078
10	9.636	592.5	13.272	0.151
50	49.619	2957.0	66.269	0.731
100	99.841	5916.5	134.05	1.451

3.4 Q3 Results — GPU With Unified Memory

Table 5: GPU kernel time (ms) — `cudaMallocManaged`

K (M)	CPU (ms)	1blk/1thr (ms)	1blk/256thr (ms)	N-blks/256thr (ms)
1	0.714	59.2	1.323	0.020
5	4.374	295.8	6.643	0.079
10	9.636	591.4	13.318	0.152
50	49.619	2957.6	67.071	0.729
100	99.841	5915.3	134.35	1.453

3.5 Q4 — Performance Charts

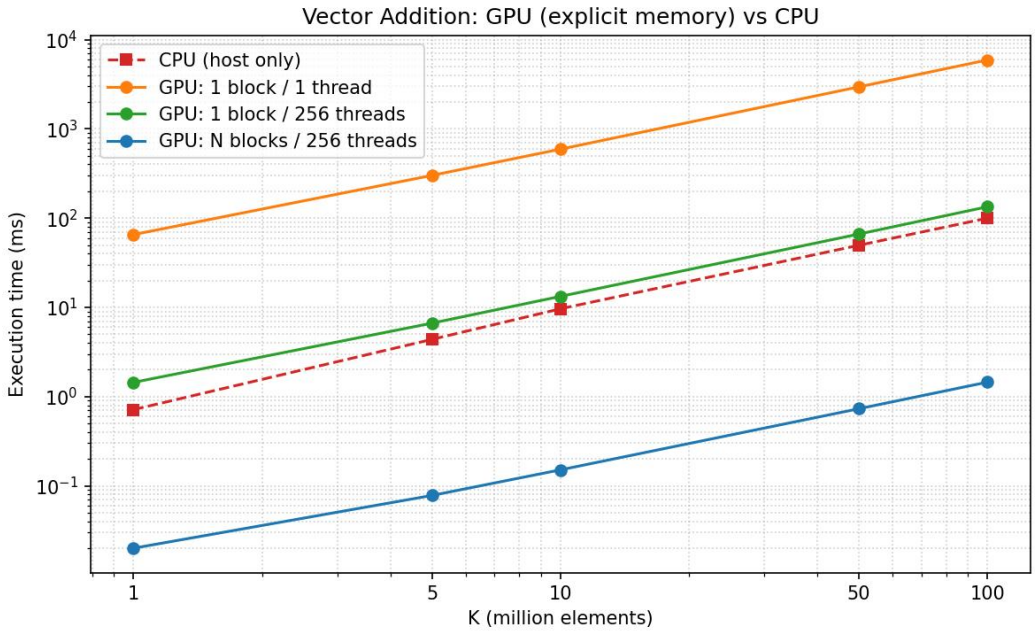

Figure 1: Vector addition time vs. K — GPU (explicit memory) vs. CPU (log–log scale)

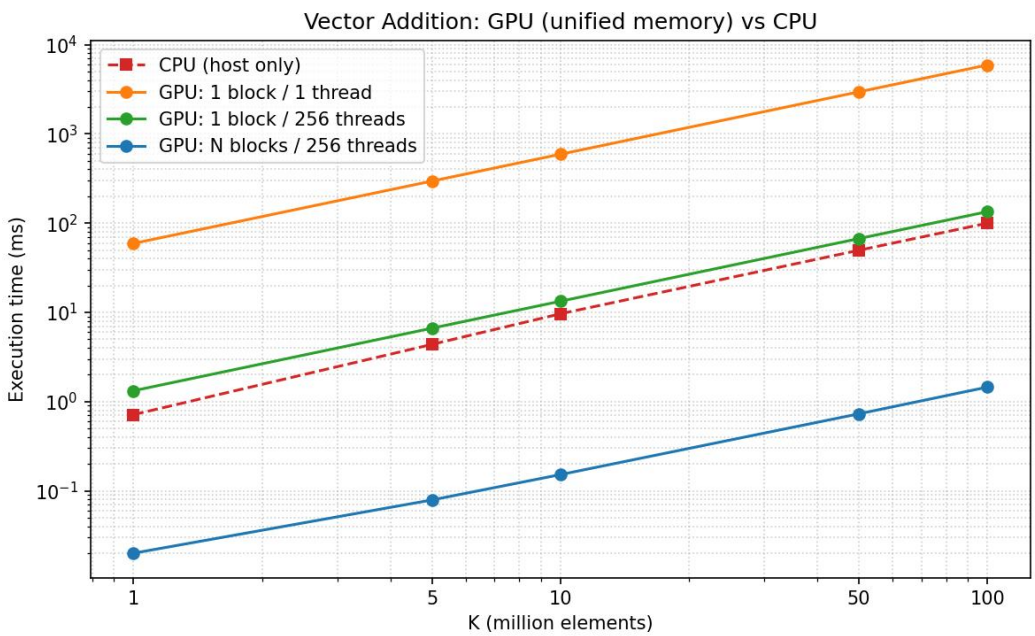

Figure 2: Vector addition time vs. K — GPU (unified memory) vs. CPU (log–log scale)

3.6 Analysis

Single-thread GPU is about 60× slower than the CPU at $K = 1$. Each memory request stalls for hundreds of cycles with no other warps to switch to, so HBM latency is fully exposed. N-blocks/256-threads ends up 68× faster than the CPU at $K = 100$.

Unified memory and explicit memory come out basically the same in kernel time. Migration happens in the warm-up pass so the data is already sitting on the GPU by the time the timed run starts.

4 Part-C: Convolution in CUDA (50 points)

4.1 Problem Setup

- Input I: $3 \times 1024 \times 1024$, $\quad I[c, h, w] = c(w + h) \quad$ (double)
- Filter F: $64 \times 3 \times 3 \times 3$, $\quad F[k, c, i, j] = (c + k)(i + j) \quad$ (double)
- Padded input I_0: zero-pad by $P = 1$, giving $3 \times 1026 \times 1026$
- Output O: $64 \times 1024 \times 1024$
- True convolution (filter is flipped):

$$O[k, h, w] = \sum_{c} \sum_{fh} \sum_{fw} F[k, c,\, FH - 1 - fh,\, FW - 1 - fw] \cdot I_0[c,\, h + fh,\, w + fw]$$

4.2 C1 — Simple Convolution (no shared memory)

One thread per output element (k, h, w). Each thread loops over (c, fh, fw) and reads directly from global memory:

```
for (int c = 0; c < C; c++)
  for (int fh = 0; fh < FH; fh++)
    for (int fw = 0; fw < FW; fw++) {
        double f  = F[k*C*FH*FW + c*FH*FW + (FH-1-fh)*FW + (FW-1-fw)];
        double i0 = I0[c*H0*W0 + (h+fh)*W0 + (w+fw)];
        val += f * i0;
    }
```

Listing 4: C1 kernel inner loop

Not much to it. Each thread handles one output element, loops over the full (c, fh, fw) space, reads filter and input values straight from global memory each time through.

4.3 C2 — Tiled Convolution (shared memory)

Each 16×16 thread block handles a 16×16 output patch for a single output channel. For each input channel, the block cooperatively loads an 18×18 input halo (`TILE + FH - 1` on each side) into shared memory, then each thread computes its output value from shared memory instead of global memory.

The whole point is that neighboring output elements pull from overlapping parts of the input, so loading one shared region for the whole block should be cheaper than each thread re-reading from global memory on its own.

4.4 C3 — cuDNN Convolution

I used `CUDNN_DATA_DOUBLE` and `CUDNN_CONVOLUTION` mode. I had `CUDNN_CROSS_CORRELATION` at first, which computes the same operation but without flipping the filter — my checksums were off from C1 and C2 by a significant margin and I spent longer than I'd like to admit figuring out why. Switching to `CUDNN_CONVOLUTION` fixed it immediately.

Algorithm was selected via `cudnnFindConvolutionForwardAlgorithm`, which picked algorithm 0 (`IMPLICIT_GEMM`) with no extra workspace needed.

4.5 C4 — OpenAI Triton (Bonus)

Implemented in `c4.ipynb`. The kernel is launched on a 3-D grid `(C_OUT, ceil(H/16), ceil(W/16))`; each program instance handles a 16×16 output tile for one output channel, loading filter and input tiles via `tl.load` with boundary masks and storing results with `tl.store`. Timing uses `torch.cuda.Event` with a warm-up pass.

4.6 Program Output (`program_output.csv`)

Line	Contents
C1	`122756344698240.000000,4.395`
C2	`122756344698240.000000,5.552`
C3	`122756344698240.000000,1.772`
C4	`2.818`

4.7 Performance Comparison

Table 6: Convolution performance on V100 — $1 \times 3 \times 1024 \times 1024$ input, $64 \times 3 \times 3 \times 3$ filter

Implementation	Time (ms)	Checksum	Speedup vs C1
C1 — Simple CUDA	4.395	122756344698240	1.00×
C2 — Tiled CUDA	5.552	122756344698240	0.79×
C3 — cuDNN	1.772	122756344698240	**2.48**×
C4 — Triton (bonus)	2.818	(assert_close passed)	1.56×

4.8 Analysis

C2 being slower than C1 is the one result I kept second-guessing. My initial reaction was that I'd made a bug somewhere and ran the checksum comparison a couple times before accepting the timing. The checksums match, so the math is right, it's just slow.

The problem is that a 3×3 filter is too small for tiling to pay off. The halo is 18×18 (324 elements) for a 16×16 (256-element) output tile, so over a third of what you load is overlap. On top of that there are two `__syncthreads()` barriers per input channel, which adds up across all three channels. The global memory savings just don't outweigh that overhead for this specific filter size. If the filter were 7×7 or 11×11 the math would look very different. Also worth noting: for a 3×3 filter the GPU's L2 cache probably handles a lot of the input reuse that shared memory is supposed to buy you, so C1 benefits from that indirectly.

cuDNN at 2.48× — algorithm 0 (`IMPLICIT_GEMM`), no extra workspace, backed by NVIDIA's optimized GEMM code.

Triton at 1.56× is decent for something written in Python. The JIT output isn't as optimized as cuDNN but it's not far off C1 either.

All three checksums match exactly, which was also how I confirmed C2 wasn't broken — I'd been staring at the C2 timing wondering if I'd made an indexing error.

5 Conclusion

Coalesced access gave 13–26% here, which is on the lower end of what the theory predicts.

The Part-B single-thread result is what I'd point someone to if they asked why parallelism matters on a GPU. It's not just slower than the fully-parallel case — it's slower than the CPU, by a lot, because memory latency on HBM has nowhere to hide when you only have one thread issuing requests. Going from 1-block/256-threads to N-blocks/256-threads is the same idea: one active SM vs. all 80 of them.

C2 was slower than C1, which I would probably predict wrong again without actually running it. The 3×3 filter is small enough that the L2 already covers most of the input reuse, and the tiling overhead ends up costing more than the global reads it was supposed to save. cuDNN was 2.48× over C1 and needed no extra workspace for algorithm 0.